

Problem 1: Self assessment.

s_1	s_2	s_3
s_1	s_2	s_3

q_a	1	s_3
s_1	$\perp$	s_3

s_1	q_a	1
q_b	s_1	$\perp$

s_1	s_2	q_a
s_1	q_b	s_2

s_1	s_2	s_3
s_1	s_2	q_b

$$s_1, s_2, s_3 \in \{0, 1, \perp\}$$

In my solution, I mistakenly overlooked the configuration where the 1 is positioned on the far left of the 2x3 window, resulting in only five cases instead of six. This omission affects the completeness of the enumeration. To fix this, I need to carefully re-examine the layout possibilities and explicitly list them all. This ensures a fully rigorous and correct solution.

Problem 2: Approximation warm-up.**(a)**

Suppose you have an instance of a minimization problem whose optimal value is 6. Specify the range of values that a $3/2$ -approximation algorithm could output when run on this instance.

For a minimization problem with an optimal value of 6, a $\frac{3}{2}$ -approximation algorithm guarantees that the solution it produces will be no more than $\frac{3}{2}$ times the optimal value. That means the worst-case output is:

$$\frac{3}{2} \times 6 = 9.$$

Since the best-case scenario is that the algorithm finds the optimal solution, the output could be as low as 6. Thus, the algorithm could output any value in the range $[6, 9]$.

(b)

Suppose you have an instance of a maximization problem whose optimal value is 25. Specify the range of values that a $2/5$ -approximation algorithm could output when run on this instance.

With the same reasoning as above we get the output could be any value in the range $[10, 25]$

Problem 3: Revisiting Times Square.

$$\text{K-PATH} = \left\{ (G = (V, E), s, t, k) : \begin{array}{l} G \text{ has a simple path from } s \text{ to } t \text{ that} \\ \text{visits at least } k \text{ vertices in } V \end{array} \right\}.$$

(a)

Show that K-PATH is NP-hard.

Proof.

Wts: $\text{HAMCYCLE} \leq \text{K-PATH}$

Define $f : \Sigma^* \rightarrow \Sigma^*$ s.t. $f(G = (V, E)) \rightarrow (G' = (V', E'), v, v', |V| + 1)$

WLOG let $V = \{v_1, \dots, v_n\}$. Define $V' = V \cup \{v'\}$.

Let $v = v_1$ and define E' s.t. $E \subset E'$ and if $(v, u) \in E$ then $(v', u) \in E'$ for $u \in V$.

Correctness:

- If G has a Hamiltonian Cycle, then $(G', v, v', |V| + 1) \in \text{K-PATH}$:

Suppose G has a Hamiltonian cycle. That is, there is a cycle

$$(v_1, v_{i_2}, v_{i_3}, \dots, v_{i_n}, v_1)$$

that visits every vertex exactly once. In G' , we can form a simple path from v to v' by following the cycle from v_1 through the other vertices:

$$v_1 \rightarrow v_{i_2} \rightarrow \dots \rightarrow v_{i_n}$$

and then, because v' has the same outgoing edges as v_1 , we know there is an edge from the last vertex v_{i_n} to v' (since in the original cycle there is an edge from v_{i_n} to v_1). This produces a simple path of length $|V| + 1$ (counting vertices $v_1, v_{i_2}, \dots, v_{i_n}, v'$). Hence, $(G', v, v', |V| + 1) \in \text{K-PATH}$.

- If $(G', v, v', |V| + 1) \in \text{K-PATH}$, then G has a Hamiltonian Cycle:

Assume there exists a simple path in G' from v to v' of length $|V| + 1$. Because G' has $|V| + 1$ vertices in total and the path is simple, it must include every vertex in V exactly once (with v' appearing only at the end). Denote this path as:

$$v_1 \rightarrow v_{j_2} \rightarrow \dots \rightarrow v_{j_n} \rightarrow v'.$$

Notice that all edges on the segment $v_1 \rightarrow v_{j_2} \rightarrow \dots \rightarrow v_{j_n}$ lie in E (since $E \subset E'$)

and thus form a simple path in G that visits every vertex in V . Finally, because v' was defined to have the same outgoing edges as v_1 , the edge from the second-to-last vertex v_{j_n} to v' implies that there is an edge from v_{j_n} to v_1 in G . Therefore, by adding the edge (v_{j_n}, v_1) , we complete a cycle that visits every vertex exactly once—i.e., a Hamiltonian cycle in G .

Runtime:

The transformation will only take $O(|G|)$ to copy G into G' then it will take $O(|E|)$ to add all the new vertices for v' . Thus, we get $O(|G|)$ this is polynomial in respect to the input.

Therefore, $\text{HAMCYCLE} \leq_p \text{K-PATH}$ ■

(b)

Given an efficient decider D_{KP} for K-PATH, design and analyze (both correctness and runtime) an efficient algorithm that given $(G = (V, E), s, t)$, and k^* , the length of the longest path from s to t , finds an actual simple path from s to t that goes through k^* vertices. The path should be returned as a set of edges (the edges do **not** have to be ordered). You may assume that $\{s, t\} \subseteq V$. If no such path exists, output $\emptyset$.

Proof.

■ This algorithm outputs the simple path from s to t using k^* vertices.

Algorithm 1 ReconstructLongestPath($G = (V, E), s, t, k^*$)

Require: A graph $G = V, E$, two vertices $s, t \in V$ and k^* the number of vertices in a longest simple path from s to t

Ensure: A set $P \subset E$ of edges representing a simple path from s to t using k^* vertices, or $\emptyset$ if no such path exists

```

1: if ( $k^* > |V|$ ) then
2:   return  $\emptyset$ 
3:  $P \leftarrow \emptyset$  ▷ Initialize the set of edges in the path.
4:  $Visited \leftarrow \{s\}$ 
5:  $current \leftarrow s$ 
6:  $rem \leftarrow k^*$  ▷ Remaining vertices to include (including the current one).
7: while  $rem > 1$  do ▷ While the path is not complete.
8:    $found \leftarrow \text{false}$ 
9:   Let  $G' = G[V \setminus Visited]$  ▷ Restrict to unvisited vertices.
10:  for all  $v \in V$  do
11:    if ( $(current, v) \in E$  or  $(v, current) \in E$ ) and  $v \notin Visited$  and
      DKP( $G', v, t, rem - 1$ ) returns YES then
12:       $P \leftarrow P \cup \{(current, v)\}$ 
13:       $Visited \leftarrow Visited \cup \{v\}$ 
14:       $current \leftarrow v$ 
15:       $rem \leftarrow rem - 1$ 
16:       $found \leftarrow \text{true}$ 
17:      break ▷ Proceed to next iteration of while loop.
18:  if  $found$  is false then
19:    return  $\emptyset$  ▷ No valid continuation found.
20: return  $P$ 

```

Correctness

If the algorithm returns P

1. Initialization: The algorithm starts at s with $Visited = \{s\}$ and sets $rem = k^*$. Thus, initially, the intended path will include s and have a total of k^* vertices.
2. While-Loop Invariant: In each iteration of the while loop, the algorithm does the following:
 - It restricts the graph to the unvisited vertices by computing $G' = G[V \setminus Visited]$.

- It iterates over all vertices v (or more precisely, those such that there is an edge $(current, v)$ or $(v, current)$ and $v \notin Visited$).
- For each candidate v , it calls DKP on $(G', v, t, rem - 1)$. This DKP call asks whether there is a simple path from v to t that uses exactly $rem - 1$ vertices.
- If DKP returns YES, it means there is a continuation from v to t that, when appended to the current partial path, yields a path using exactly $rem - 1$ more vertices. The algorithm then appends the edge $(current, v)$ to P , marks v as visited, updates $current$ to v , and decrements rem .

At the beginning of each iteration, there is a simple s -to- $current$ path that visits exactly $k^* - rem + 1$ vertices, and the algorithm has ensured that there is a simple continuation from $current$ to t that uses the remaining rem vertices since we return P at the end.

3. Termination: When $rem = 1$, the loop terminates. At that point, the path built (represented by P) is a simple path from s to $current$ with exactly k^* vertices. Because the DKP calls guarantee that each step has a valid continuation to t , the final vertex $current$ must be t (or else the DKP would have returned NO at some point). Thus, the set P forms a simple s -to- t path using exactly k^* vertices.

If there exists a simple path from s to t with k^* vertices

1. Suppose there is a simple path P^* from s to t that uses exactly k^* vertices.
2. In the first iteration, because P^* is simple, there is a neighbor v of s along P^* (and $v \notin Visited$) such that the remainder of P^* (from v to t) is a simple path using exactly $k^* - 1$ vertices. Thus, the DKP call on $(G', v, t, k^* - 1)$ returns YES.
3. The algorithm will choose this v , add (s, v) to P , update $Visited$ and continue.
4. By an inductive argument, in each iteration the correct choice (the next vertex on P^*) will satisfy the DKP check, and the algorithm will eventually reconstruct P^* .

Therefore, if a simple s -to- t path with k^* vertices exists, the algorithm will find it and return the corresponding set of edges P .

Runtime

If $k^* > |V|$ then the runtime is $O(1)$.

Thus, now assume that $k^* \leq |V|$

The outerloop takes $O(k^*)$

The innerloop takes $O(|V|)$.

The creation of G' takes $O(|G|)$

DKP takes $O(|G|^c)$ for some $c \in \mathbb{N}$.

Thus, the total runtime is $O(k^*(|G| + |V||G|^c))$ which is polynomial with respect to G since $k^*, |V| < |G|$. Thus, the algorithm is efficient. ■

Problem 4: Finding the colors through SAT.

$$3\text{-COLORING} = \{G : G \text{ is a graph and is 3-colorable}\}$$

$$3\text{SAT} = \{\phi : \phi \text{ is a satisfiable Boolean formula}\}$$

Proof.

Algorithm 2 Find3Coloring($G = (V, E)$)

Require: A graph $G = (V, E)$

Ensure: A set $C \subseteq V \times \{r, g, b\}$ representing a 3-coloring of G , or $\emptyset$ if none exists.

```

1:  $\phi \leftarrow f(G)$  ▷  $f$  is the function from the PTMR above
2: if  $D_{3\text{SAT}}(\phi)$  returns NO then
3:   return  $\emptyset$ 
4:  $C \leftarrow \emptyset$  ▷ Coloring: set of vertex-color pairs.
5: for all  $v \in V$  do
6:   for all  $c \in \{r, g, b\}$  do
7:     Let  $\phi_{v=c}$  be the formula  $\phi$  s.t. we set  $c_i$  be true, so we can collapse all the
       clauses that are now true.
8:     if  $D_{3\text{SAT}}(\phi_{v=c})$  returns YES then
9:        $C \leftarrow C \cup \{(v, c)\}$ 
10:      Update  $\phi \leftarrow \phi_{v=c}$  ▷ Fix the color for  $v$  for future queries.
11:      break ▷ Proceed to next vertex.
12: return  $C$ 

```

If the algorithm outputs C :

then at every step, when the algorithm fixes a color for vertex v_i based on a YES from $D_{3\text{SAT}}$, it is guaranteed that there exists a satisfying assignment (i.e. a valid 3-coloring) that extends the current partial assignment since we assume it outputs C . Thus, the final assignment is consistent and satisfies the constraints of the 3-coloring encoding.

If G is 3-colorable:

then the initial formula ϕ is satisfiable since G is 3 colorable if and only if $f(G) = \phi \in 3\text{SAT}$. Then, by trying each color for each vertex, at least one color choice for every vertex will lead to a satisfiable formula. Hence, the algorithm will eventually assign a color to every vertex.

No False Positives:

If the graph is not 3-colorable then the initial $D_{3\text{SAT}}(\phi)$ call will return NO, and the algorithm correctly outputs $\emptyset$.

Runtime:

f takes $O(|V| + |E|)$

D_{3SAT} takes $O((|V| + |E|)^c)$ for $c \in \mathbb{N}$.

Both loops take $O(3|V|) = O(|V|)$

Inside the loop the making $\phi_{v=c}$ takes $O(|V| + |E|)$ since there are $O(|V| + |E|)$ clauses thus we check everyone and remove it if c_i is in it.

Thus, again D_{3SAT} takes $O(|V| + |E|)^c$ for $c \in \mathbb{N}$ since $\phi_{v=c}$ in the worse case is size $O(|V| + |E|)$.

Then the rest are $O(1)$ operations.

Thus overall $O(|V| + |E| + (|V| + |E|)^c + |V|((|V| + |E|)^c + |V| + |E|)) = O(|V|(|V| + |E|)^c)$.

Thus, this is polynomial with respect to $|G|$. Thus, this is efficient. ■

Problem 5: Approximate f -SET-COVER.

In this question, you will consider a variant of the SET-COVER problem where each element of the universe is in a limited number of subsets. Formally, in the f -SET-COVER problem, we are given a “universe” (set) U and subsets $S_1, \dots, S_n \subseteq U$ where each universe element appears in at most f of the subsets. The goal is to find a *smallest* number of the subsets that “covers” U , i.e., an $I \subseteq \{1, \dots, n\}$ such that $\bigcup_{i \in I} S_i = U$. We assume that $\bigcup_{i=1}^n S_i = U$, otherwise no solution exists.

(a)

Prove that $I(u) \cap I(u') = \emptyset$ for every distinct $u, u' \in E$. In other words, prove that if u and u' are each selected as the uncovered element in different iterations, then no S_i has both u and u' .

Proof.

Assume u, u' are distinct and are each selected as the uncovered element in different iterations.

Assume for contradiction that $I(u) \cap I(u') \neq \emptyset$. Thus, there is a S_i s.t. $u, u' \in S_i$.

WLOG let u be picked at an earlier iteration than u' . Therefore, this implies that we will add S_i to C .

However, this is a contradiction since we would not have picked u' at a later step since it is not in $U \setminus C$ anymore. Thus, $I(u) \cap I(u') = \emptyset$. ■

(b)

We want a lower bound on $\text{OPT} = |I^*|$. Using the previous part, prove that $|E| \leq |I^*|$.

Proof.

Assume for contradiction that $|E| > |I^*|$. This would imply that we picked more elements as the uncovered element than there were indices in the optimal cover. This implies that by the pigeonhole principle that at least two $u, v \in E$ have to share the same index in I^* . However, proven above in part (a) this cannot happen.

Thus, $|I^*| \leq |E|$ ■

(c)

We want an upper bound on $\text{ALG} = |I|$. Prove that $|I| \leq f \cdot |E|$, and conclude that the f -cover algorithm is an f -approximation algorithm for the f -SET-COVER problem.

Proof.

By definition, we know that $|I(u)| \leq f$.

Also since,

$$I = \bigcup_{u \in E} I(u)$$

and proven in part (a) that all the $I(u)$ are disjoint this implies that

$$|I| = \sum_{u \in E} |I(u)| \leq \sum_{u \in E} f = f|E|$$

Therefore, by part (b), $|I| \leq f|E| \leq f|I^*|$.

Thus, the f -cover algorithm is an f -approximation algorithm. ■

(d)

Prove that for every positive integer f , there is an input for which the f -cover algorithm necessarily outputs a cover that is f times as large as an optimal one.

Proof.

Let $f \in \mathbb{Z}$. Let $U = \{x\}$. Let $S_1 = \{x\}$, and $S_i = \{x\}$ for $i \in \{2, \dots, f\}$.

So first we pick x then we will add $1, \dots, f$ to I . Thus, $|I| = f$, but the optimal solution is 1. ■

Problem 6:

$$\text{ALLOCATE} = \left\{ (n, C = \{c_1 \dots c_m\}, k) : C \text{ can be partitioned into } n \text{ subsets} \right. \\ \left. \text{where each subset sums to at most } k \right\}.$$

(a)Prove that ALLOCATE $\in$ NP.*Proof.*

Let the certificate be an array of m pairs of integers s.t. the first number is bounded by m and the second by m . Each pair takes $\log(n) + \log(m)$ spaces. Thus, our certificate takes up $O(m \log(n) + m \log(m))$ which is polynomial with respect to the input which is $O(m \log c + \log n + \log k)$ for $c = \max\{c_1, \dots, c_m\}$.

Algorithm 3 Verify Certificate for ALLOCATE

Require: A certificate $V = \{v_1, \dots, v_m\}$, item counts $C = \{c_1, \dots, c_m\}$, number of cashiers n , and workload limit k

Ensure: Accept if the certificate is valid, reject otherwise

```

1: if  $n \geq m$  then
2:   for  $i \in \{1, \dots, m\}$  do
3:     if  $c_i > k$  then return reject
4:   return accept
5:  $S[1, \dots, n] \leftarrow [0, 0, \dots, 0]$ 
6: for  $i \in \{1, \dots, m\}$  do
7:    $S[v_i] \leftarrow S[v_i] + c_i$ 
8: for  $i \in \{1, \dots, n\}$  do
9:   if  $S[i] > k$  then return reject
10: return accept

```

Runtime:

If $n \geq m$ then the run time takes $O(m)$

If $n < m$ then the run time takes $O(2n + m) = O(m)$.

Thus, in both cases we get $O(m)$ which is polynomial with respect to the input.

Correctness:

In general if $c_i > k$ then $(n, C, k) \notin \text{ALLOCATE}$ this is because there would be no where to place this customer without going over the item limit.

If $(n, C = \{c_1, \dots, c_m\}, k) \in \text{ALLOCATE}$:

If $n \geq m$ the one valid partition of C would be to make each customer go into their own lane since each customer has less than k items as shown above. Thus, $S_1 = \{c_1\}, S_2 = \{c_2\}, \dots, S_m = \{c_m\}, S_{m+1} = \emptyset, \dots, S_n = \emptyset$.

Thus, since we can always make this partition when $n \geq m$ we only have to check if $c_i \leq k$. Thus, since this is the case when we are in Allocate. We know the algorithm will accept on line 4 with any certificate.

If $n < m$ then now we now there is a valid partition $S_1, \dots, S_n$ s.t. the sum of elements in S_i is at most k . Thus, define $V = \{v_1, \dots, v_m\}$ s.t. $v_i = j$ if $c_i \in S_j$.

Now what the algorithm now does is sum all the c_i, c_j together in S if $v_i = v_j$. This is the same as summing the partition set S_{v_j} . Then we check later that each $S[i] < k$. Since we know that each S_i the sum is less than k . We know that $S[i]$ will be too. Thus, the algorithm will accept on line 10.

If there exists a certificate V s.t. the verifier will accept (n, C, k)

If it accepted on line 4. Then we know that $n \geq m$ and that each $c_i > k$ so similar argument to above we know that we can partition this so each customer is in their own lane. Thus, $(n, C, k) \in \text{ALLOCATE}$.

If it accepted on line 10 then consider the partition $S_j = \{c_{i_1}, \dots, c_{i_r}\}$ s.t. $v_{i_k} = j$.

Shown by the program since we know that the sum of $c_{i_1} + \dots + c_{i_r} < k$ we know that this applies for each S_j . Thus, $S_1, \dots, S_n$ is a valid partition s.t. each S_j will sum to at most k . ■

(b)

Prove that $\text{SUBSET-SUM} \leq_p \text{ALLOCATE}$

Proof.

Define $f : \Sigma^* \rightarrow \Sigma^*$ s.t. S is the sum of all elements in A .

$$f(A, s) = \begin{cases} (2, A \cup \{S - 2s\}, S - s) & \text{if } 2s \leq S, \\ (2, A \cup \{2s - S\}, s) & \text{otherwise.} \end{cases}$$

Runtime:

It takes $|A|$ to sum all the elements to get the value S .

Then it takes $|A|$ to copy A , and the rest are $O(1)$ operations.

Thus, the runtime is overall $O(|A|)$ which is polynomial with respect to the input.

Correctness:

If $(A, s) \in \text{SUBSET-SUM}$ then there is a subset S_1 s.t. the sum of S_1 sums to s .

If $2s \leq S$ (the sum of A):

Let cashier 1 get $S_1 \cup \{S - 2s\}$ and let cashier 2 get $A \setminus S_1$.

Thus, 1 gets $s + S - 2s = S - s$ and 2 gets $S - s$. Thus, $(2, A \cup \{S - 2s\}, S - s) \in \text{ALLOCATE}$

If $S < 2s$:

Let cashier 1 get S_1 and let cashier 2 get $A \setminus S_1 \cup \{2s - S\}$.

Thus, 1 gets s and 2 gets $S - s + 2s - S = s$. Thus, $(2, A \cup \{2s - S\}, s) \in \text{ALLOCATE}$.

If $f(A, s) \in \text{ALLOCATE}$ then :

If $2s \leq S$ then $(2, A \cup \{S - 2s\}, S - s) \in \text{ALLOCATE}$

Then since the total sum of $A \cup \{S - 2s\}$ is $S + S - 2s = 2S - 2s$. This means that each cashier will get exactly $S - s$.

Thus, since one cashier gets the element $S - 2s$ we now that for some collection S_1 that $S_1 + S - 2s = S - s \implies S_1 = s$. Thus, $(A, s) \in \text{SUBSET-SUM}$.

If $S < 2s$ then $(2, A \cup \{2s - S\}, s) \in \text{ALLOCATE}$

Then since the total sum of $A \cup \{2s - S\}$ is $S + 2s - S = 2s$. This means that each cashier will get exactly s . Thus, take the cashier with the collection that does not have the element $\{2s - S\}$. Thus, that is a collection of elements of A that sum to s .

Thus, $(A, s) \in \text{SUBSET-SUM}$.

Therefore, $\text{SUBSET-SUM} \leq_p \text{ALLOCATE}$ ■

(c)

Letting OPT be the optimal workload, prove that $\text{OPT} \geq \max\{c_1, c_2, \dots, c_m\}$.

Proof.

Let $M = \max\{c_1, \dots, c_m\}$. Since one cashier line will have M this requires that the workload must already at least M to fit it. Since if it was less than M then there would be no where for M to go. Thus, $M \leq \text{OPT}$ ■

(d)

Prove that if $m \leq n$, GREEDYALLOCATE necessarily returns an optimal allocation.

Proof.

If $m \leq n$ then this implies that the greedy allocate will allocate all customers to their own cashier. Thus, the computed allocation for this solution will be $M = \max\{c_1, \dots, c_m\}$. Since we know from part (c) that $M = \text{ALG} \leq \text{OPT}$ this implies that $\text{ALG} = \text{OPT}$. ■

(e)

Prove that $\text{OPT} \geq \frac{1}{n} \cdot \sum_{j=1}^m c_j$.

Proof.

Since there is a way to partition the items to each cashier has a maximum of OPT items. This implies that the total amount of items scanned by all n of the cashiers is at most $n \cdot \text{OPT}$.

Thus:

$$\sum_{j=1}^m c_j \leq n \cdot \text{OPT} \implies (1/n) \sum_{j=1}^m c_j \leq \text{OPT}$$

(f)

Prove that $\text{ALG} \leq 2 \cdot \text{OPT}$, where ALG is the workload obtained by GREEDYAL-LOCATE.

Proof.

Let j be the cashier that ends up with the max load. Let the load of cashier j right before the last customer was assigned to j be L , and let c be the number of items of the last customer.

Thus, $\text{ALG} = L + c$.

Since the algorithm is greedy we know that cashier j has the smallest number of items before adding c . Thus, we know every other cashier had at total of L .

Thus, $n \cdot L \leq \sum_{i=1}^m c_i$.

By part (e)

$$L \leq (1/n) \sum_{j=1}^m c_j \leq \text{OPT}$$

By part (a) since $c \leq \text{OPT}$

$$\text{ALG} = L + c \leq (1/n) \sum_{j=1}^m c_j + \text{OPT} \leq 2\text{OPT}$$

Problem 7:

An algorithm runs in *pseudo-polynomial* time if its runtime is polynomial in the numeric value of the input (e.g. C), rather than the bit-length of the input (e.g. $\log C$). We have seen such algorithm in the algorithm unit, namely, the dynamic programming algorithm for the 0-1 knapsack problem, which runs in $O(nC)$ time.

Let Π be an optimization problem. An algorithm A is called an *approximation scheme* for Π if, for every instance $I \in \Pi$ and every error parameter $\epsilon > 0$, it outputs a solution $A(I)$ satisfying the following *approximation guarantee*:

- $A(I) \geq (1 - \epsilon)\text{OPT}(I)$ if Π is a maximization problem.
- $A(I) \leq (1 + \epsilon)\text{OPT}(I)$ if Π is a minimization problem.

We call A a *polynomial-time approximation scheme (PTAS)* if, for every fixed $\epsilon > 0$, its runtime is polynomial in the input size $|I|$. If the runtime is polynomial in **both** $|I|$ and $1/\epsilon$, we call it a *fully polynomial-time approximation scheme (FPTAS)*.

(a)

Prove that FPTAS-KNAPSACK is a fully polynomial-time approximation scheme for the 0-1 knapsack problem. In particular, justify both the correctness of the approximation guarantee and the runtime of the algorithm.

Proof.

First we can assume that $W[i] \leq C$ for all i .

This is because if we assume that this algorithm is a *FPTAS* which we are trying to prove then consider this example.

$V = \{100, 1\}$, $W = \{10, 1\}$, $C = 1$, $\epsilon = 1/2$.

Thus, $\text{ALG} = 0$ and $\text{OPT} = 1$. However, this contradicts $0 \geq 1/2(\text{OPT}) = 1/2$.

Thus, we can assume $W[i] \leq C$ for all i . This implies that $V_{\max} \leq \text{OPT}$ since if it was not then consider the knapsack with just $V_{\max}$ in it this would be a solution larger than OPT , thus contradiction.

Let S^* be an optimal solution for the original value with total value $\text{OPT} = \sum_{i \in S^*} V[i]$

Let S be the computed solution with value $\text{ALG} = \sum_{i \in S} V'[i]$.

For each i we have $\frac{V[i]}{k} - 1 < V'[i] \leq \frac{V[i]}{k} \implies V[i] - k < kV'[i] \leq V[i]$.

Thus, for S^* of items we have:

$$\sum_{i \in S^*} V[i] = \text{OPT} - k|S| < \sum_{i \in S^*} V'[i] \leq \sum_{i \in S^*} V[i] = \text{OPT}$$

Since $k|S^*| \leq k \cdot n = \epsilon V_{\max} \leq \epsilon \text{OPT}$ Since $k = (\epsilon V_{\max} / n)$ and $V_{\max} \leq \text{OPT}$

$$\text{OPT} - \epsilon \text{OPT} < \sum_{i \in S^*} V'[i] \leq \text{OPT}$$

Since the algorithm outputs the max value for the scaled values we get that $\sum_{i \in S^*} V'[i] \leq \text{ALG}$

Therefore,

$$\text{OPT} - \epsilon \text{OPT} < \sum_{i \in S^*} V'[i] \leq \text{ALG}$$

Thus,

$$\text{ALG} \geq (1 - \epsilon) \text{OPT}$$

Runtime:

This runs in $O(n + T(DP))$, but DP runs in $O(n \times V'_{\text{total}})$ and $V'_{\text{total}} \leq n \times (V_{\max} / k) = n^2 / \epsilon$. Which leads to a run time of $O(n^3 / \epsilon)$

Thus, the total runtime is $O(n + n^3 / \epsilon) = O(n^3 / \epsilon)$.

Thus, *FPTAS – KNAPSACK* is a fully polynomial-time approximation scheme. ■

(b)

Prove that if $P \neq NP$, then no polynomial time approximation algorithm for the knapsack problem can guarantee

$$|\text{ALG} - \text{OPT}| \leq k$$

for any fixed constant $k > 0$, where OPT is the maximum total value achievable by an optimal solution and ALG is the total value returned by the approximation algorithm.

Proof.

Assume $P \neq NP$.

Assume for contradiction that there exists a polynomial-time approximation algorithm

A for the knapsack problem s.t.

$$|\text{ALG} - \text{OPT}| \leq k$$

for $k > 0$.

Consider the decision problem of knapsack which is NP-complete. Consider the decider:

Input: (V, W, T, C) where the algorithm finds if there is a subset of items of total weight at most C whose total value is at least T .

First scale V s.t. $V' = (k + 1)V$. All elements multiplies by $k + 1$

Compute $A(V', W, C) = \text{ALG}$ in polynomial time.

If $\text{ALG} \geq T(k + 1)$ then return "accept"

else return "reject"

Correctness:

Assume that (V, W, T, C) is in KNAPSACK. Then by assumption we know that for $A(V', W, C) = \text{ALG}$ that $|\text{OPT} - \text{ALG}| \leq k$. However, since OPT and ALG are multiples of $k + 1$. This implies there difference is too. Thus, $\text{OPT} = \text{ALG}$ since there is no multiple of $k + 1$ in $[0, k]$ other than 0.

Thus, we know that the actual optimal solution is $\text{ALG}/(k + 1)$. Thus, since we know that $\text{ALG}/(k + 1) \geq T \implies \text{ALG} \geq (k + 1)T$. Thus, we would accept (V, W, T, C) .

If our algorithm accepted (V, W, T, C) then this would imply that $\text{ALG} \geq (k + 1)T$, but since by a similar proof to above we know that $\text{OPT} = \text{ALG}$ thus we know that the actual optimal solution is $\text{ALG}/(k + 1)$. Thus, $(V, W, T, C) \in \text{KNAPSACK}$.

Also the algorithm halts on all inputs since A halts on every input, and the rest is all for loops or $O(1)$ operations.

Correctness

If $(V, W, T, C) \in \text{KNAPSACK}$

Because there is a subset S with total original value $\geq T$, in the scaled instance we have a subset S of total value

$$\sum_{i \in S} v'_i = \sum_{i \in S} ((k + 1)v_i) = (k + 1) \sum_{i \in S} v_i \geq (k + 1)T.$$

Thus, the optimal solution in the scaled instance satisfies

$$\text{OPT} \geq (k + 1)T.$$

The approximation guarantee says $|\text{ALG} - \text{OPT}| \leq k$. But both ALG and OPT are multiples of $(k + 1)$. Since every v'_i is a multiple of $(k + 1)$. The only way ALG can

differ from OPT by at most k (but remain a multiple of $(k + 1)$) is if $\text{ALG} = \text{OPT}$. Thus $\text{ALG} = \text{OPT} \geq (k + 1)T$. Our decider checks whether $\text{ALG} \geq (k + 1)T$. Since $\text{ALG} \geq (k + 1)T$ is indeed true, the decider accepts.

If our decider accepts (V, W, T, C)

Suppose the decider accepts, which means we found $\text{ALG} \geq (k + 1)T$. As before, ALG and OPT are multiples of $(k + 1)$, and $|\text{ALG} - \text{OPT}| \leq k$. The only possibility is $\text{ALG} = \text{OPT}$. Thus $\text{OPT} \geq (k + 1)T$ in the scaled instance. It follows there exists a feasible subset S in the scaled instance with

$$\sum_{i \in S} v'_i = (k + 1) \sum_{i \in S} v_i \geq (k + 1)T.$$

Dividing by $(k + 1)$, we get $\sum_{i \in S} v_i \geq T$. So S is a feasible solution in the *original* instance with total value at least T . Hence $(V, W, T, C) \in \text{KNAPSACK}$.

Runtime

Our only expensive subroutine is calling A , which (by assumption) is polynomial-time. Scaling item values, checking $\text{ALG} \geq (k + 1)T$, and returning accept or reject are all polynomial (or linear) steps.

Hence the decider runs in polynomial time overall.

Also, the decider is a decider since it halts on every input since there is only for loops, and A which halts on every input.

Since, the decision problem for Knapsack 0 – 1 is NP-complete, and we showed there is a polynomial algorithm that decides it this implies that $P = NP$ which is a contradiction.

Thus, there is no algorithm A . ■